

스파르타 Spring 3기
CH5 플러스 스프링 프로젝트

티켓을 JAVA라



ticket-javara.site

팀
깃허브



CONTENTS

1 팀 소개 및 프로젝트 소개

2 전체 시스템 구조

3 인프라

4 이벤트 & 검색

5 예매

6 실시간 채팅 (CS)

7 회고

8 Q&A



팀 소개 및
프로젝트
소개



팀원 소개



T1 덤보 | 임하은

발표자
인증 / 보안
유저 도메인
프론트엔드



유재석 | 강태훈

QA
쿠폰 도메인
채팅 도메인



유희관 | 정태규

팀장
프로젝트 총괄
인프라



박효신 | 선경안

QA
이벤트 도메인
검색/캐싱
인기 검색어



카리나 | 이지민

부팀장 및 서기
좌석/예매 도메인
결제 연동
프론트엔드



불꽃 | 카리스마

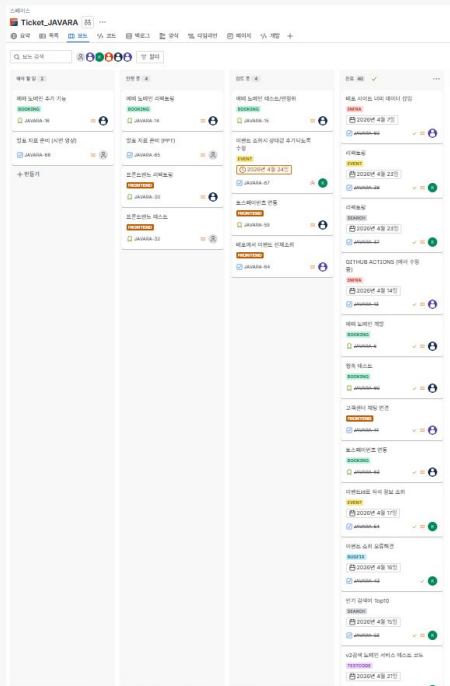
마스코트 민호



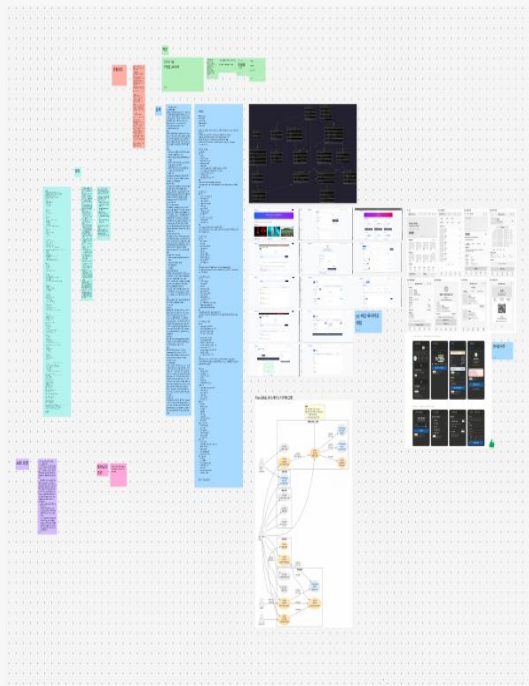
마일스톤



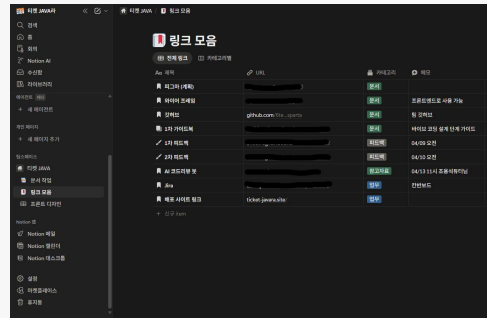
협업 도구



지라



피그마



회의록

플러스 Spring 주차

날짜 없음(1) [뒤](#) [앞](#) [홈](#) [목록](#) [새로 만들기](#) [삭제](#)

2026년 4월

일	월	화	수	목	금	토	일
29	30	31	4월 1일	2	3	4	
	월 11:30 토 20:30	월 20:30	월 20:30	토 [원칙1].. 토 20:10	토 20:30		
5	6	7	8	9	10	11	
	토 20:30	토 [원칙1].. 월 20:30	토 [원칙1].. 토 20:30	토 10:45 토 12:30 토 [원칙1].. 토 11:30 토 [원칙1].. 토 ERD이해	월 10:30 토 [원칙1].. 토 [원칙1].. 토 11:00 수학 토 15:00		
				토 [원칙1].. 토 [원칙1]..			
12	13	14	15	16	17	18	
토 [원칙1] 스터디 토 20:30							
	토 10:00 토 [원칙1].. 토 10:30	토 [원칙1].. 토 [원칙1]..		토 [원칙1].. 토 11:10	토 [원칙1].. 토 [원칙1]..		

노션

깃허브 구성

The screenshot shows the GitHub interface for the organization '6team-Plus-project-sparta'. The top navigation bar includes links for Overview, Repositories (4), Projects (1), Packages, Teams, People (6), Insights, and Settings. A search bar is located on the right. The left sidebar contains filters for repository visibility: All, Contributed by me, Admin access, Public, Private, Sources, Forks, Archived, and Templates. The main content area displays a list of repositories, with the first two highlighted by a red box:

- Ticket_JAVARA** (Public)
 - Language: Java
 - Stats: 0 forks, 0 stars, 0 issues, 0 pull requests
 - Updated: 1 minute ago
- ticket-javara-frontend** (Public)
 - Description: 티켓 JAVA라 팀 프로젝트 프론트엔드 저장소
 - Language: TypeScript
 - Stats: 0 forks, 0 stars, 0 issues, 0 pull requests
 - Updated: 6 hours ago

Below the highlighted repositories, there are two more entries:

- docs** (Public)
 - Description: 티켓을 JAVA라 플랫폼의 문서/기획/설계 자료를 관리하는 레포지토리입니다. 팀 철학, 시스템 아키텍처, 요구사항 정의, 기술 문서 등을 포함합니다.
 - Stats: 0 forks, 0 stars, 0 issues, 0 pull requests
 - Updated: 4 days ago
- .github** (Public)
 - Description: HOT6의 공통 설정, 팀 가이드, Organization 프로필을 관리하는 레포지토리입니다.
 - Stats: 0 forks, 0 stars, 0 issues, 0 pull requests
 - Updated: 3 weeks ago

At the bottom, a ProTip! message states: "When viewing an organization's repositories, you can use the `props:` filter to filter by custom property."

시스템 구성 요소

기술 스택



Java 17



Spring boot 3.5.13



배포 인프라

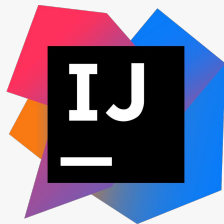


외부 API



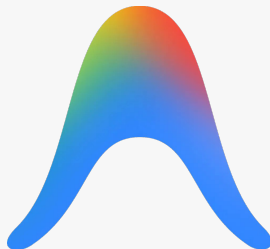
개발 도구

백엔드



IntelliJ IDEA

백엔드

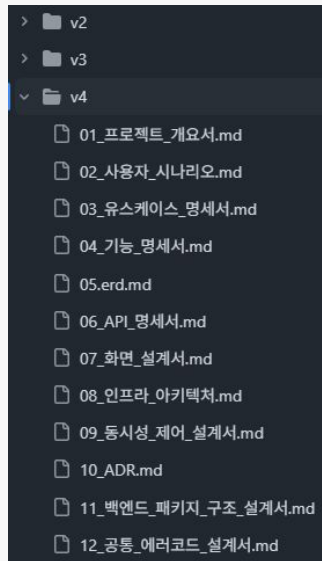


Google Antigravity

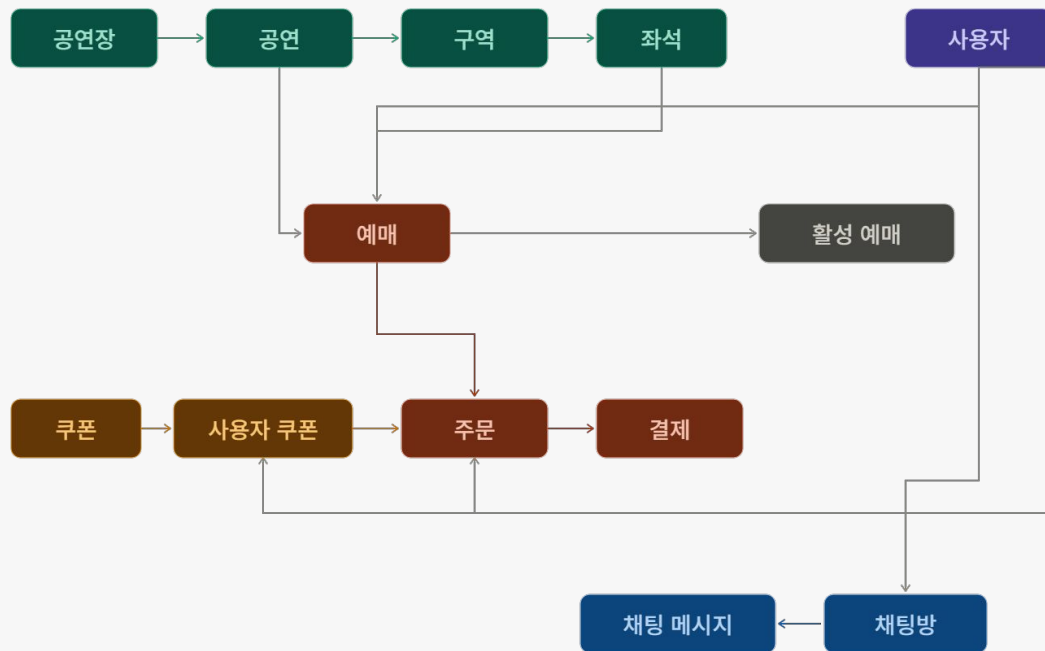
프론트 엔드



KIRO



ERD



■ 공연장/구역/좌석

■ 예매/주문/결제

■ 사용자

■ 쿠폰

■ 채팅

■ 할성 예매



스파르타 Spring 3기

티켓을 JAVA라



ticket-javara.site

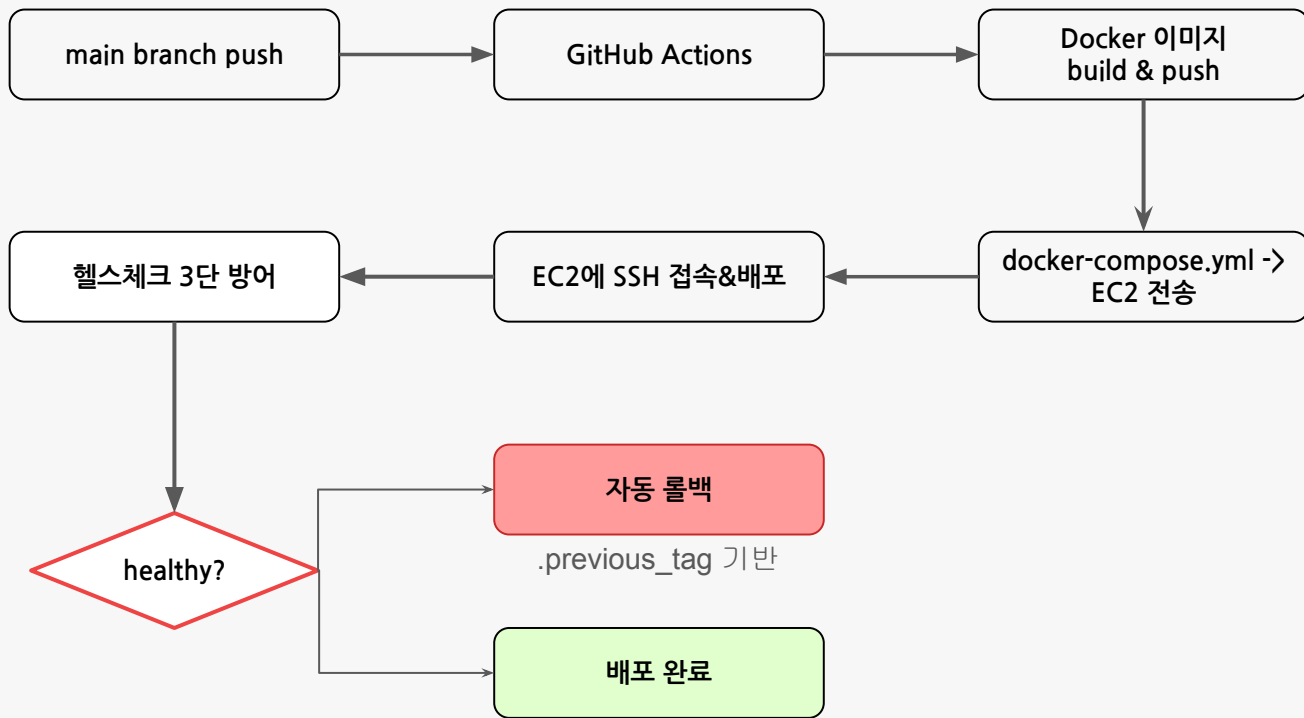
TEAM HOT6 시연영상





각 도메인 별
핵심 포인트

인프라 - 이 파이프라인 구조를 선택한 이유 - 1

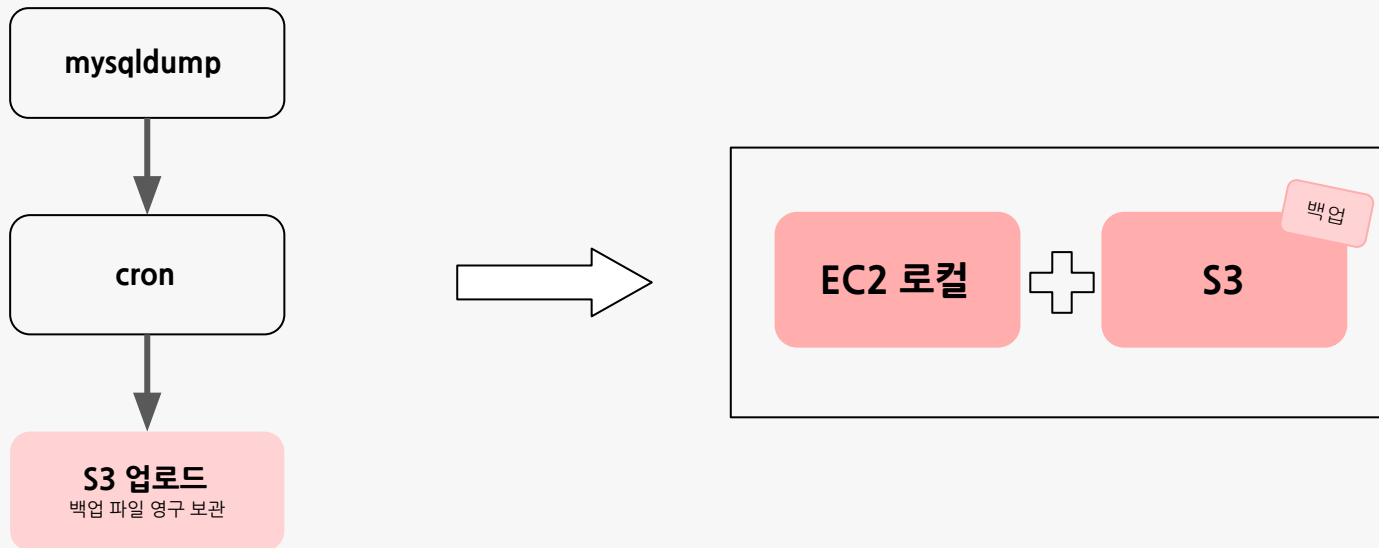


인프라 - 이 파이프라인 구조를 선택한 이유 - 2

항목	Amazon ECR	Docker Hub 현재 사용중
비용 (소규모)	유료 (소량 무료)	무료 플랜 있음
세팅 난이도	IAM 설정 필요	간단 · 즉시 사용
AWS 연동	네이티브 통합	별도 인증 필요
보안	IAM · KMS · VPC	기본 수준
Pull 속도 (EC2)	빠름 (같은 리전)	보통
Rate Limit	없음	200회 / 6시간
현재 프로젝트 적합도	오버스펙	적합

● 강점 ● 보통 ● 상대적 우위

인프라 - 백업을 설정한 이유



이벤트 - 왜 DELETED를 별도 status로 뒀는지

조회
불가

복구
불가

구현
복잡도

DELETE

Soft Delete

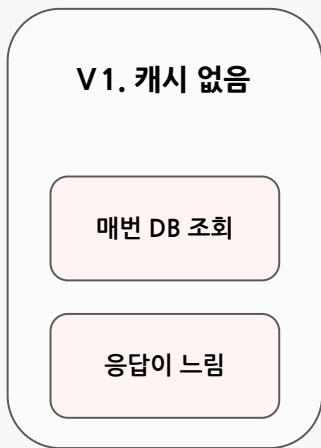
FK 보호

안전장치

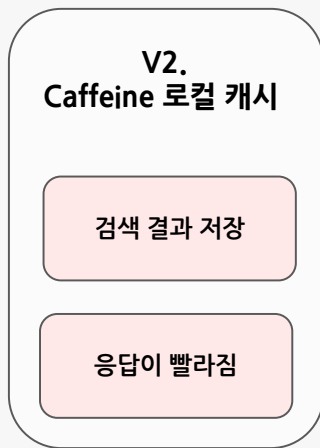
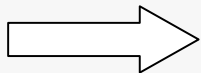
조회
단순화

검색 - 캐싱 3단계 진화 이유 - 1

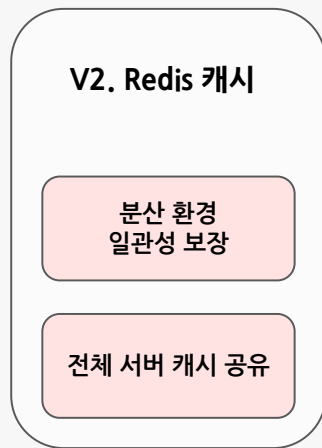
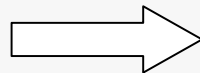
왜 3단계로 나뉘었나? — 각 단계의 한계를 직접 경험해야 다음 단계의 필요성이 생긴다



한계: 매 요청마다 MySQL Full Scan



한계: Scale-out 시 서버 간 캐시
불일치



한계: Redis 왕복 1~2ms 추가 비용
발생

검색 - 캐싱 3단계 진화 이유 - 2(v1)

※ 벌크데이터: 이벤트 약 5000개

HTTP TicketJavara > v1검색

GET http://localhost:8080/api/v1/events/search?category=CONCERT

Docs Params Authorization Headers 7 Body Scripts Settings

Body Cookies Headers 11 Test Results 200 OK 52 ms 2.98 KB

{ } JSON Preview Visualize

```
1 {
2   "content": [
```

검색 - 캐싱 3단계 진화 이유 - 2(v2 Caffeine 로컬 캐시)

HTTP TicketJavara > v2검색

GET <http://localhost:8080/api/v2/events/search?category=CONCERT> V2 여러번 검색

Docs Params Authorization Headers 7 Body Scripts Settings

Query Params

Key	Value	Description
✓ category	CONCERT	
Key	Value	Description

Body Cookies Headers 12 Test Results 200 OK 8 m

Key	Value
X-Cache	HIT

검색 - 캐싱 3단계 진화 이유 - 2(v2 Redis)

HTTP TicketJavara > v2검색

v2 : Redis 캐시
히트

GET http://localhost:8080/api/v2/events/search?keyword=아이유

Docs Params Authorization Headers 7 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

1

Body Cookies Headers 15 Test Results 200 OK 64 ms 2.0

Key	Value
Vary	Origin
Vary	Access-Control-Request-Method
Vary	Access-Control-Request-Headers
X-Cache	HIT

검색 - Feature Flag 전환 전략

Feature Flag

코드 변경
Zero

lock과 대기 구조

즉시 롤백 가능

```
# application.yml
```

```
cache:
```

```
  provider: caffeine # → redis 로 변경만으로 전환
```

```
# application.yml
```

```
lock:
```

```
  provider: lettuce # → redisson 로 변경만으로 전환
```



예매 - 왜 Lettuce SETNX를 선택했는지 - 1

낙관적 락

장점

DB 락 없음 → 성능 좋음
충돌 적은 환경에 적합

단점

충돌 시 재시도 필요
트래픽 많으면 재시도 폭발

비관적 락

장점

데이터 정합성 확실
충돌 발생 시 안전 (순차 처리)

단점

대기 발생 (Blocking)
DB 커넥션 점유 → 병목 위험

분산락

장점

Fail Fast (즉시 실패) → 트래픽에 강함
여러 서버에서도 동시성 제어 가능

단점

Redis 의존성 증가
직접 구현 시 복잡 (TTL, 해제 처리)

예매 - 왜 Lettuce SETNX를 선택했는지 - 2

Lettuce SETNX

장점

단순함, 빠름, Redis만 있으면 됨

단점

TTL, 락 해제 Lua Script, 예외 처리 등을 직접 잘 구현해야 함

Redisson RLock

장점

구현 편함, tryLock() 같은 API 제공, 락 관리 기능 풍부

단점

라이브러리 의존성 증가, 내부 동작 이해 필요

ZooKeeper

장점

정합성 강함, 분산 시스템 조율에 강함

단점

구축/운영 복잡, 프로젝트 규모에 비해 무거움

핵심 판단 기준

티케팅 = 충돌 많음 + Fail Fast 필요 → Lettuce SETNX가 최적

예매 - Lettuce SETNX를 어떻게 구조화 했는지 - 1

Lettuce SETNX 구조화

Lua Script 원자화

GET이 DEL 사이에 다른 스레드가 끼어드는 것을 원천 차단 필요

→ UUID 검증 + DEL을 한 번의 스크립트로 처리

HoldLockFacade 분리

@Transactional 메서드 안에서 락을 해제하면 COMMIT 전에 락이 풀리는 버그 발생

→ Facade를 별도 Bean으로 분리해서 트랜잭션과 락의 생명주기를 독립적으로 관리

락 획득 순서 고정

데드락을 방지 필요

락 순서 고정
좌석 Hold 락 → 쿠폰 사용 락
→ DB 트랜잭션

본 원칙은 Hold 영역에 우선 적용. OrderService 쿠폰 락은 v2에서 동일 패턴으로 확장 예정

예매 - Lettuce SETNX를 어떻게 구조화 했는지 - 2

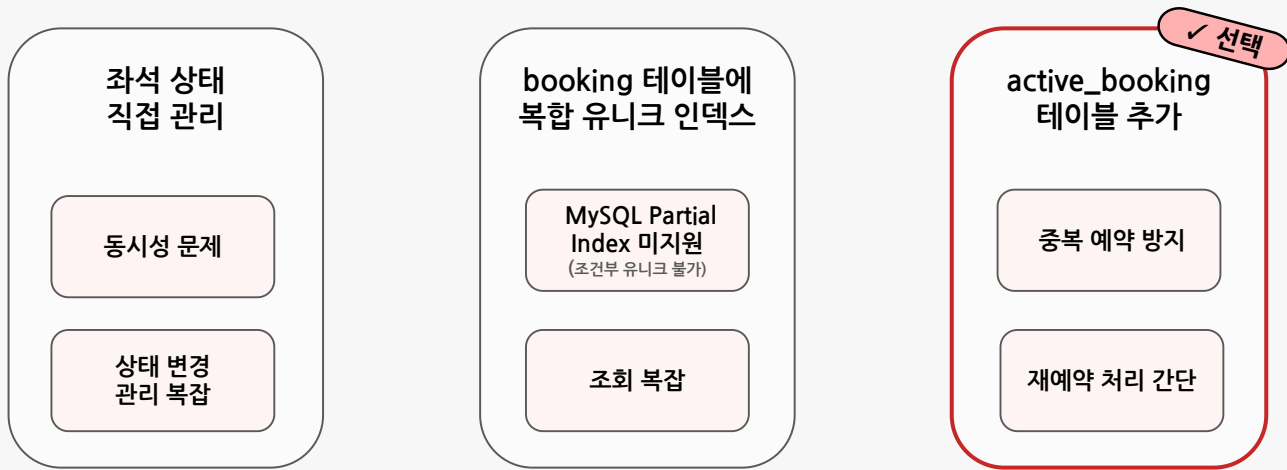
흐름

SETNX 락 획득 → Facade 실행
→ HoldService 비즈니스 로직 → Lua Script 락
해제

검증 : 스레드(100건) 동시 Hold 테스트

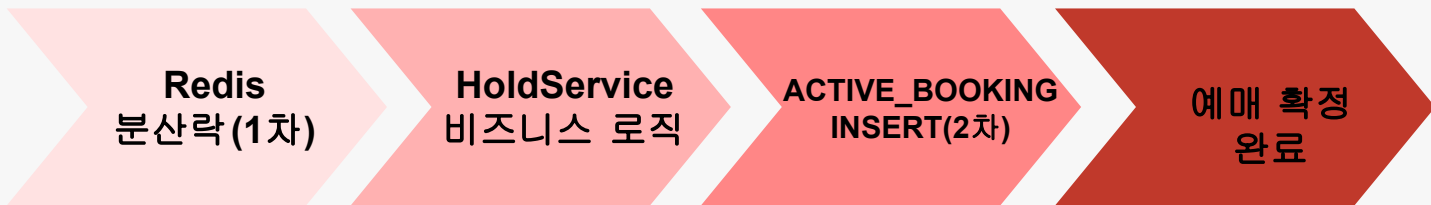
- 락 없음 → 다수(99건) 성공 X
- 락 적용 → 정확히 1건만 성공
(ExecutorService + CyclicBarrier로 100개 스레드 동시
실행)

예매 - ACTIVE_BOOKING 테이블을 따로 둔 이유 - 1



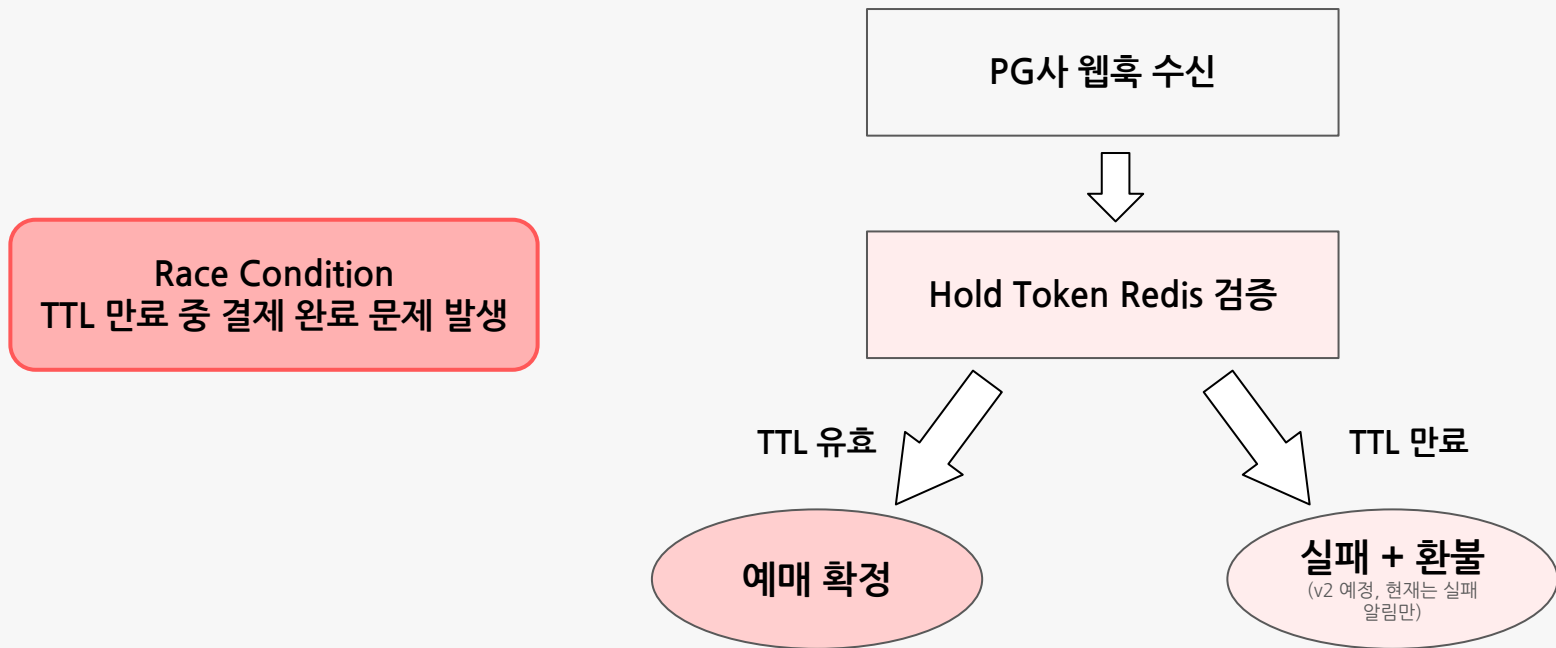
두 번째 INSERT → PK 충돌로 즉시 실패 = DB 레벨 물리적 차단

예매 - ACTIVE_BOOKING 테이블을 따로 둔 이유 - 2



1차 방어선(Redis 분산락) + 2차 방어선(ACTIVE_BOOKING PK) → Redis 장애에도 DB가 막음

예매 - Hold TTL 만료 후 웹훅 수신을 왜 이렇게 처리했는지



Toss cancel API 연동은 v2 — 현 MVP는 결제 검증/실패 처리에 우선 집중

채팅 - WebSocket + STOMP를 선택했는지

순수 WebSocket

※ 라우팅•세션 관리를 직접 구현해야 하는 부담

외부 메시지 브로커

Redis Pub/Sub, RabbitMQ 등

✗ 단일 서버에서 불필요한 네트워크 오버헤드 (오버엔지니어링)

Spring 내장 STOMP

✓ 선택

✓ 인메모리 기반 빠른 성능
✓ @MessagingAdapter 직관적 라우팅

📌 미래 대비: `RedisChatPublisher/Subscriber + RedisChannelConfig` 사전 구현 완료
– Scale-out 시 브로커 설정 변경만으로 다중 서버 대응 가능



개인 회고
및
예상 질문

개인 회고

정태규

"배포는 올리는 게 아니라
설정하는 것이다"

이번 프로젝트에서는 기본적으로 인프라를 주로 개발에 참여하였습니다. 처음에는 그저 ec2에 프로젝트를 컴파일해서 올리는 것이라고 생각했는데 환경 변수 등록, 워크플로, AWS콘솔 (클라우드프론트, ECS, ECR) 등 설정해야 할 부분이 많았습니다. 개발이 진행되면서 워크플로, AWS에서의 경험을 통해 혼자서도 개발 + 배포를 할 수 있는 기반을 다진 것 같습니다.

이지민

"AI 시대일수록
원리 이해가 답이다"

이번 프로젝트는 AI를 적극적으로 활용하는 바이브 코딩 방식으로 진행했는데, AI가 코드를 제안해줘도 결국 트랜잭션 격리 수준이나 락의 생명주기 같은 원리를 이해하지 못하면 왜 에러가 나는지 파악조차 할 수 없다는 걸 직접 느꼈고, 백엔드 로직이 완성돼도 프론트와의 타임존 차이나 API 연동 포인트에서 예상치 못한 버그가 생긴다는 점에서 팀 전체가 시스템을 함께 이해하는 것이 얼마나 중요한지 배웠습니다.

선경안

"설계가 탄탄해야
AI도 제대로 쓴다"

이벤트와 캐싱부분을 맡아 프로젝트에 제일 기반에 해당하는 기능을 구현한 느낌이 들었습니다. SDD라는 명세기반 바이브코딩을 진행하면서 설계에 중요성과 AI가 코드를 짜주더라도 결국 책임과 결정은 인간이 하는 것이기에 지속적인 공부가 필요함을 느꼈습니다. 검색기능을 구현하면서 v1, v2로 전환하고 각 캐싱에 성능을 눈으로 직접보며 phase에 따른 기술 선택에 중요성을 깨닫게 되었습니다.

임하은

"AI는 도구,
공부는 여전히 내 몫이다"

인증/보안과 유저도메인 담당하였고, 바이브 코딩을 적극적으로 활용하며 속도가 빠르고 지시가 정확하면 결과도 어느정도 잘 나온다는 것을 알게 되어 유익했습니다. 또한 프롬프트를 준비하는 과정에서는 모든 도메인에 대한 정책과 기능에 대해 인지하고 있어야 한다는 것을 알 수 있었습니다. ai를 사용하여 간단하게 짧은 기간동안 프로젝트를 끝내겠다는 마음보다는 새로운 개발 도구를 사용한다는 마음으로 열심히 공부해야겠다는 생각이 들었습니다.

강태훈

"AI를 쓰려면 지시하고
검수할 실력이 먼저다"

이번 프로젝트에서는 바이브 코딩 툴을 사용하여 쿠폰과 cs 채팅 도메인을 구현하였습니다. 처음에는 ai를 활용하니 금방 끝날것이라 생각했지만 ai라고해도 모든 에러를 잡아내진 못하고 어노테이션을 빼먹는 등 실수가 많다는 것을 알 수 있었습니다. ai를 사용하더라도 그에 올바른 지시를 내릴 사람과 ai가 만들어준 결과를 검수할 사람이 필요함을 알게되었고 이에 적합한 능력을 길러야 할 필요성을 깨닫게 되었습니다.

예상 질문 - 인프라



Redis가 장애 나면 서비스 전체가 멈추나요?

기능별로 다르게 봐야 합니다.
인기 검색어 같은 부가 기능은 **Redis** 장애 시 빈 배열을 반환하고
검색 자체는 정상 동작하게 할 수 있습니다.
반면 좌석 **Hold**나 쿠폰 선착순 발급처럼 **Redis** 의존도가 높은 기능은 정합성이 중요하므로
무리하게 진행하지 않고 실패 응답을 주는 것이 안전합니다.
핵심 정합성이 필요한 기능과 부가 기능을 분리해서 대응합니다.



Redis를 쓰면 MySQL만 쓸 때보다 복잡도가 늘어나는데 감수한
이유는 뭔가요?

맞습니다. **Redis**를 사용하면 운영 복잡도와 장애 대응 포인트가 늘어납니다.
하지만 티켓팅에서는 임시 좌석 점유, 분산락, 인기 검색어, 캐시처럼 **Redis**가 적합한 영역이
많았습니다.
특히 TTL 기반 **Hold**와 빠른 **Fail Fast** 동시성 제어는 **MySQL**만으로 처리하면 **DB** 부하와 대기
시간이 커질 수 있습니다.
그래서 **Redis**는 정합성의 최종 저장소라기보다, 빠른 임시 상태 처리와 트래픽 완충 역할로
사용했습니다.



예상 질문 - 이벤트&검색



ENDED랑 DELETED가 둘 다 "더 이상 안 보이는 상태"인데
왜 분리했나요?

사용자 화면에서 둘 다 안 보일 수 있지만, 의미와 관리 목적이 다릅니다.
ENDED는 “정상 종료된 기록”, **DELETED**는 “삭제 처리된 데이터”라서
백오피스, 통계, 감사 로그, 데이터 복구 관점에서 구분이 필요합니다.



QueryDSL을 왜 썼나요? JPA @Query로도 되지 않나요?

@Query로도 가능하지만, 저희 검색은 조건이 계속 선택적으로 붙는 동적
쿼리였습니다.

QueryDSL을 쓰면 조건이 있을 때만 조립할 수 있고,
문자열 기반 JPQL보다 타입 안정성이 좋아서 유지보수에 유리했습니다.

그래서 **단순 조회는 JPA**로 처리하고,
검색처럼 **조건 조합이 많은 부분은 QueryDSL**로 분리했습니다.



예상 질문 - 예매



Redis 락이 있는데 왜 DB의 **ACTIVE_BOOKING** 테이블까지 두었나요?

Redis 락은 1차 방어선이고, DB 제약은 최종 방어선입니다.
Redis 장애, 락 TTL 만료, 구현 버그 같은 상황이 생겨도 같은 좌석이 중복 확정되면 안 됩니다.

그래서 확정된 좌석만 **ACTIVE_BOOKING**에 저장하고,
seat_id를 PK로 두어 DB 레벨에서 중복 확정을 막았습니다.
즉, Redis는 빠른 동시성 제어용이고, DB 제약은 정합성을 지키는 최후의 안전장치입니다.



락을 잡고 트랜잭션을 처리하면 되는 것 아닌가요?
왜 **Facade**를 분리했나요?

락과 트랜잭션의 생명주기가 섞이면 문제가 생길 수 있습니다.
예를 들어 **@Transactional** 메서드 안에서 락을 해제하면,
실제 **DB commit**이 끝나기 전에 락이 먼저 풀릴 수 있습니다.
그러면 다른 스레드가 아직 반영되지 않은 **DB 상태**를 읽고 들어올 위험이 있습니다.
그래서 **Facade**는 락 획득과 해제를 담당하고,
내부 **Service**는 트랜잭션 처리를 담당하도록 분리했습니다.
이렇게 하면 **DB commit** 이후에 락이 해제되도록 생명주기를 더 명확하게 관리할 수 있습니다.



예상 질문 - 채팅



JWT를 STOMP 어디서 검증하고 있나요?

URL에 토큰을 넣는 방식은 로그나 브라우저 히스토리에 토큰이 남을 수 있어서
보안상 위험하다고 판단했습니다. 또, 메시지를 보낼 때마다 토큰을 검증하고
DB를 조회하면,
채팅 메시지가 많아질수록 DB 부하가 커질 수 있습니다.
그래서 저희는 채팅에 사용자가 처음 연결할 때,
스톰프 헤더로 JWT를 전달받아 한 번만 검증하도록 설계했습니다.



다른 사용자의 채팅방을 SUBSCRIBE하면 어떻게 됩니까?

현재는 채팅방 ID만 알면 SUBSCRIBE 가능하여 아무런 제약 없이 메시지를
실시간으로 수신할 수 있습니다. 이에 대해 인지 하고 있고 v2에서
ChannelInterceptor 혹은 StompHandler에서 SUBSCRIBE 권한 검증 추가
예정입니다





ticket-javara.site

감사합니다

Team. HOT6

임하은 | 강태훈 | 정태규 | 선경안 |
이지민



팀
깃허브





ENDED랑 DELETED가 둘 다 "더 이상 안 보이는 상태"인데 왜 분리했나요?

사용자 화면에서 둘 다 안 보일 수 있지만, 의미와 관리 목적이 다릅니다.
ENDED는 "정상 종료된 기록", **DELETED**는 "삭제 처리된 데이터"라서 백오피스, 통계, 감사 로그, 데이터 복구 관점에서 구분이 필요합니다.



SOLD_OUT은 잔여 좌석이 0이면 자동 계산할 수 있는데 왜 상태로 들고 있나요?

잔여 좌석 기준으로 계산할 수도 있지만, **운영상 수동 제어가 필요**할 수 있다고 봤습니다.

예를 들어 좌석은 남아 있어도 운영자가 판매를 중단하거나, 일부 좌석을 내부 사유로 막는 경우가 있을 수 있습니다.

그래서 잔여 좌석은 실시간 계산 값으로 보고, **SOLD_OUT**은 이벤트의 판매 상태를 표현하는 관리 값으로 분리했습니다.



39



JWT를 STOMP 어디서 검증하고 있나요?

URL에 토큰을 넣는 방식은 로그인 브라우저 히스토리에 토큰이 남을 수 있어서 보안상 위험하다고 판단했습니다. 또, 메시지를 보낼 때마다 토큰을 검증하고 DB를 조회하면,

채팅 메시지가 많아질수록 DB 부하가 커질 수 있습니다.

그래서 저희는 채팅에 사용자가 처음 연결할 때, 스프링 헤더로 JWT를 전달받아 한 번만 검증하도록 설계했습니다.



QueryDSL을 왜 썼나요? JPA @Query로도 되지 않나요?

@Query로도 가능하지만, 저희 검색은 조건이 계속 선택적으로 붙는 동적 쿼리였습니다.

QueryDSL을 쓰면 조건이 있을 때만 조립할 수 있고, 문자열 기반 JPQL보다 타입 안정성이 좋아서 유지보수에 유리했습니다.

그래서 **단순 조회는 JPA로 처리**하고, 검색처럼 **조건 조합이 많은 부분은 QueryDSL로 분리**했습니다.



39

예매 - ACTIVE_BOOKING 테이블을 따로 둔 이유 - 2

ORDER_ITEM 제거

BOOKING이 order_id + original_price 직접 보유
→ JOIN 뎁스 감소,
MVP 개발 속도 유리

BOOKING = 티켓 검임

별도 TICKET 테이블 대신
ticket_code 컬럼 추가
→ ticketId = booking_id 로
단순화

이중 방어선 구조

Redis 분산락 = 1차 방어선
ACTIVE_BOOKING PK =
2차 방어선
→ Redis 장애에도 DB가
막음

예상 질문

Q1. ENDED랑 DELETED가 둘 다 "더 이상 안 보이는 상태"인데 왜 분리했나요?

Q2.

Q3. DELETED로 전환할 때 물리 삭제(DELETE 쿼리)를 안 쓴 이유가 뭔가요?

Q4. QueryDSL을 왜 썼나요? JPA @Query로도 되지 않나요?

Q5. Feature Flag가 정확히 뭔가요? 어떻게 구현했나요?

Q6. 왜 pub/sub을 적용하지 않고 구현만 해두었나요?

Q7. 다른 사용자의 채팅방을 SUBSCRIBE하면 어떻게 됩니까?

Q8.

Q9.

왜 Redisson 안 쓰고 Lettuce SETNX?

HoldLockFacade 분리 이유는?

OrderService도 동일 패턴인가요?

ECR vs Docker Hub는 왜? 다운타임은?

Blue/Green은?

Hold TTL 만료 + 결제 성공은?

Redis가 죽으면?

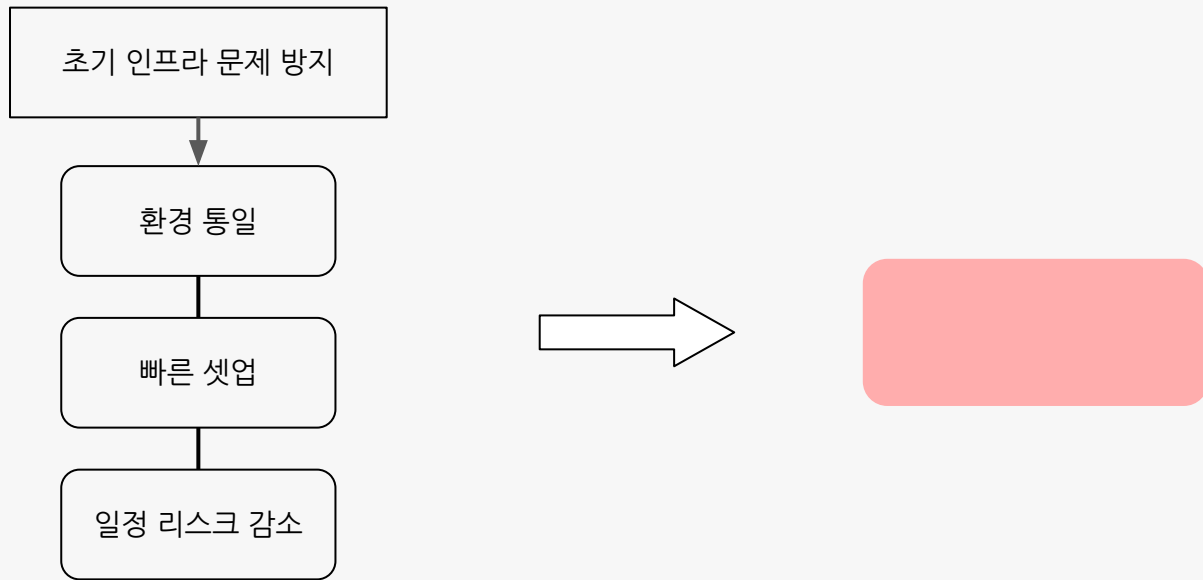
100명 동시 예약 결과 수치는?

EXPLAIN 결과 보여주세요

SUBSCRIBE 권한 재검증은?

캐시 3단계 쓰는 수치는?

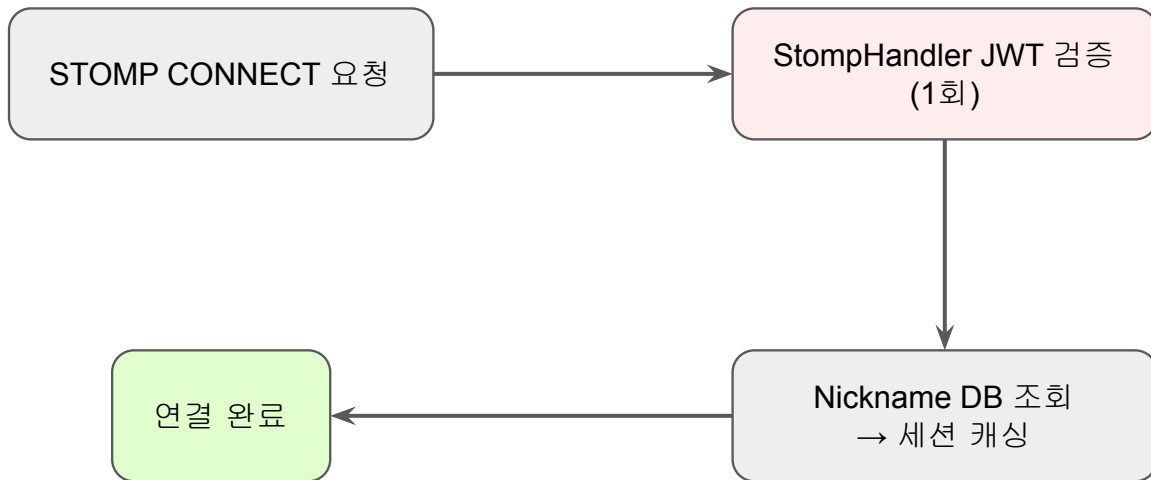
인프라 - Docker Compose 도입 이유



채팅 - JWT를 STOMP 어디서 검증하나 - 2



CONNECT 시점 처리 흐름



전체적인 흐름

(1페이지) 선택지들 -> 관련 문제

(2페이지) 우리가 선택한 선택지 -> 장점과 단점

(ppt에서는 선택지의 흐름을 보여주고 말로 장점 단점

설명

저희가 선택한 방법은 이런 흐름을 가지게 되며 이런
단점있지만 이런 장점 때문에 사용하게 됐다.)

사진 추가하기

내용 정리되면 해야 될 것들.

1. 슬라이드 페이지 번호
2. 제목 위치 통일
3. 글씨체 통일 (제목별, 내용별)
4. 전체적인 색감 맞추기
5. 글 줄이고 사진 등 이미지 활용하기
6. 필요없는 페이지는 일단 삭제하지 말고 맨 뒤로
빼놓기
7. 시연영상 소리크기 및 화질 체크
8. 오탈자 확인인

예매 - 왜 Lettuce SETNX를 골랐고 어떻게 구조화 했는지

비교 대상

Lettuce SETNX가 좋은 점

Redisson RLock 대비

구조가 단순함. **SETNX + TTL** 방식이라 동작 원리를 직접 설명하기 좋음. 라이브러리 의존성이 적고, 프로젝트 규모에 맞음.

ZooKeeper 락 대비

Redis만 있으면 되므로 인프라 추가가 필요 없음. 구축·운영 부담이 훨씬 낮음. 3주 프로젝트에 적합함.

공통 장점

빠르게 구현 가능하고, 좌석 선점처럼 “한 명만 성공, 나머지는 즉시 실패”시키는 **Fail Fast** 전략에 잘 맞음.

단점

설명

직접 구현 부담

락 획득, **TTL** 설정, 락 해제, 예외 처리 등을 직접 잘 만들어야 함.

락 해제 실수 위험

아무나 락을 지우면 안 되므로 **UUID** 값을 저장하고, **Lua Script**로 “내 락일 때만 삭제”해야 함.

고급 기능 부족

Redisson처럼 **RLock.tryLock()** 같은 편한 API, Watchdog 자동 연장 기능이 기본 제공되지 않음.

Redis 의존

Redis 장애 시 락 획득 자체가 어려워짐. 그래서 **DB PK** 같은 최후 방어선이 필요한.

채팅 - JWT를 STOMP 어디서 검증하나 - 1

보안문제

URL에 토큰 넣기

✗ 로그·히스토리 토큰 노출

성능문제

메시지 보낼 때마다
토큰 검증+DB 조회

✗ 매 메시지 DB 병목 (N+1 유사)

✓ 선택

CONNECT 때
한 번만 인증

✓ STOMP 헤더로 안전 전달
✓ 세션에 nickname 캐싱 → DB 재조회 0회

참고 자료 (개인)

예매 - 왜 Lettuce SETNX를 골랐고 어떻게 구조화 했는지

낙관적 락

충돌 안 날 거라고 믿고 일단 진행, 문제 생기면 다시 시도
미리 막지 않고, 마지막에 충돌 체크

ex

A: 결제 성공 ✓

B: "이미 팔림" → 실패 ✗ (재시도)

비관적 락 -> 요청이 살아있음

충돌 날 것 같으니까 아예 미리 막기 (DB에서)

미리 잠금,

다른 사람은 "기다리기 때문에" 안전하지만 느림

ex

A: "이거 살게요"

→ 직원: "잠깐, 이 사람 먼저 처리 중"

A 처리 중 → 다른 사람 대기

A 끝나면 → B 가능

분산락 -> 요청이 즉시 정리

여러 서버에서도 하나만 실행되게 막는 락 (Redis 같은 외부 시스템으로)

Redis 같은 외부 시스템 사용, 여러 서버에서도 동일하게 막음

다른 사람은 기다리지 않고 바로 실패

ex

A (서버1): 예약판에 이름 적기 → 성공 ✓

B (서버2): 예약판에 이름 적기 → 이미 있음 ✗

분산락이 좋은 이유

비관적 락과 다르게 기다리지 않고 바로 실패하기 때문에 트래픽
폭주에 강함

한 명만 처리

나머지는 즉시 실패

이벤트 삭제 방식 — Soft Delete

💬 **컨텍스트**: 관리자가 잘못 등록된 이벤트를 노출 중단해야 한다. 물리 삭제(DELETE) vs Soft Delete(DELETED 상태 전환)

항목	물리 삭제 (DELETE)	Soft Delete (채택)
FK 정합성	❌ BOOKING.event_id 깨짐	✅ 유지
예매 이력	❌ 삭제로 조회 불가	✅ BOOKING 이력 보존
구현 복잡도	높음 (CASCADE 처리)	낮음 (status 필터 추가)
ADR-012 철학	❌ 일관성 없음	✅ 상태 전이 방식 일관
복구 가능성	불가	가능 (status 재변경)

02 안전장치

01. FK 보호

물리 삭제 시 BOOKING.event_id FK가 깨져
예매 이력 훼손 — 절대 불가

ACTIVE_BOOKING 존재 시
DELETED 전환 차단하는 규칙 설정
(409) — 확정 예매 있으면 삭제 불가

03 조회 단순화

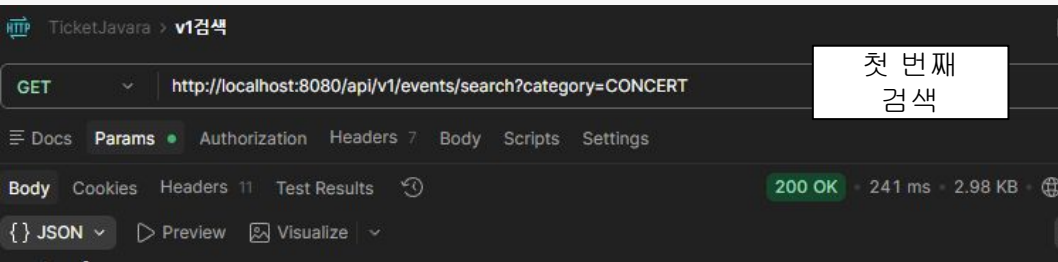
WHERE status != 'DELETED' 한 줄 추가만으로
목록·상세·좌석 조회 전부 일괄 제외

예매 - 왜 **Lettuce SETNX**를 골랐고 어떻게 구조화 했는지

락을 “안전하게” 해제 할 수 있게 -> **Lua Script**
원자화
락이랑 **DB** 작업을 분리하기 위해 ->
HoldLockFacade 분리
데드락 방지하기 위해 -> 락 획득 순서 고정

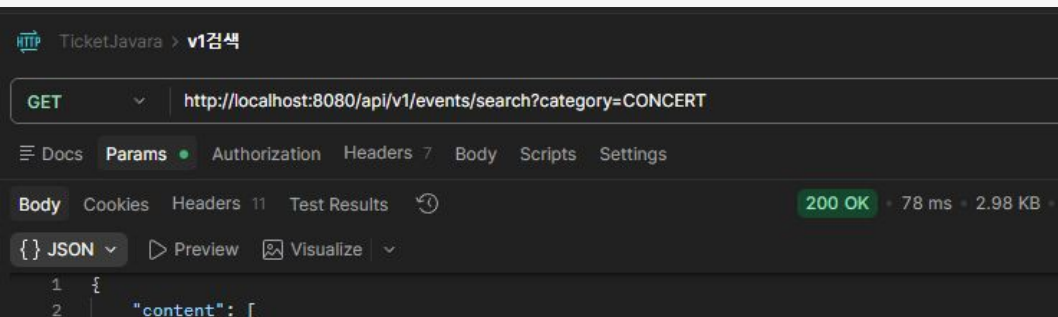
검색 - 캐싱 3단계 진화 이유 - 2(v1)

※ 벌크데이터: 이벤트 약 5000개



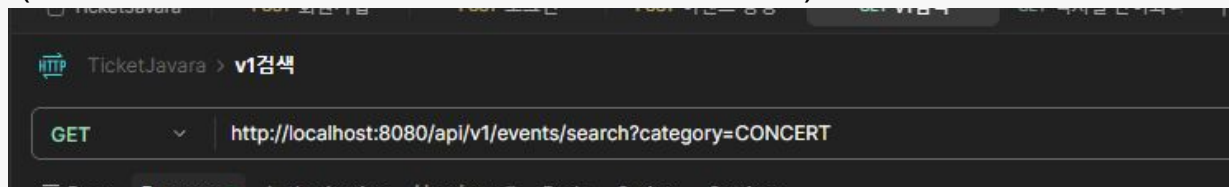
```
2026-04-14T16:38:27.201+09:00 TRACE 38156 --- [ticket-javara] [nio-8080-exec-1] o
parameter (1:BIGINT) <- [366]
2026-04-14T16:38:27.202+09:00 INFO 38156 --- [ticket-javara] [nio-8080-exec-1] c
Search] query execution time: 39 ms
```

v1콘서트 검색 첫번째



v1검색시 인텔리제이 로그 : 39ms

v1콘서트 검색 두번째: 응답속도가 확연히 차이남을 확인할 수 있다
(전 JVM웜업으로 인한 속도차이로 알고있어요.)



검색 - 캐싱 3단계 진화 이유 - 2(v2 Caffeine 로컬 캐싱)

V2 캐시Miss

HTTP TicketJava > v2검색

GET http://localhost:8080/api/v2/events/search?category=CONCERT

V2 2번째 검색

Params Authorization Headers 7 Body Scripts Settings

Query Params

Key	Value	Description
✓ category	CONCERT	
Key	Value	Description

200 OK

251 ms

Body Cookies Headers 12 Test Results

Key

X-Cache

Body Cookies Headers 12 Test Results

200 OK

15 ms

2.99 KB

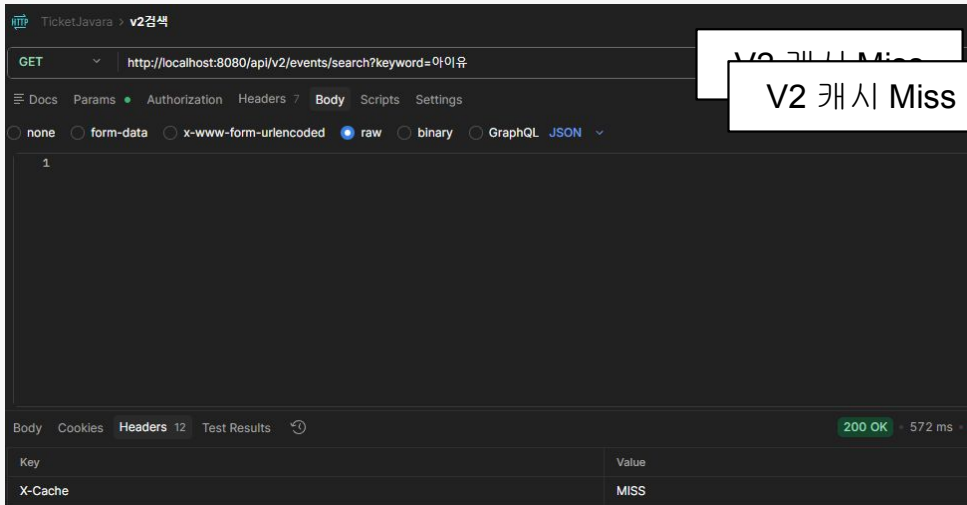
Key

Value

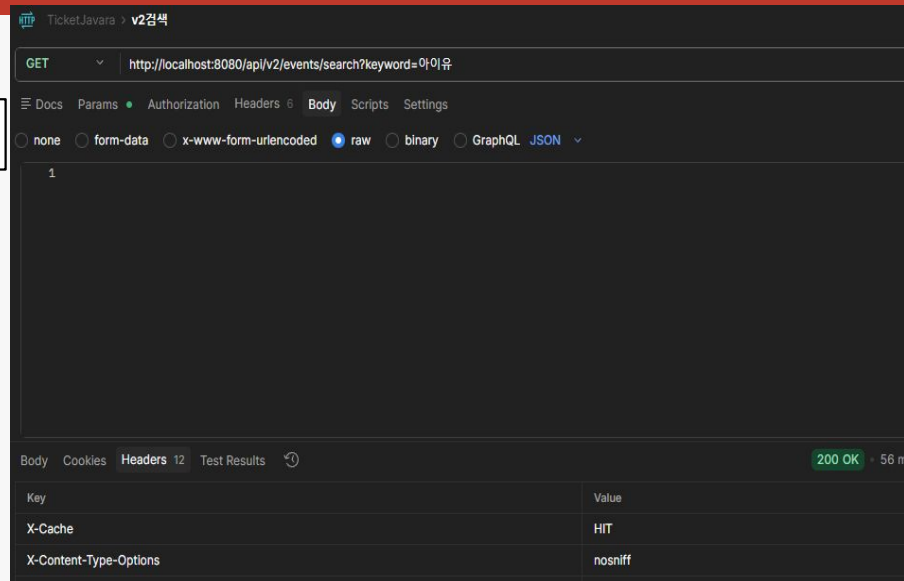
X-Cache

HIT

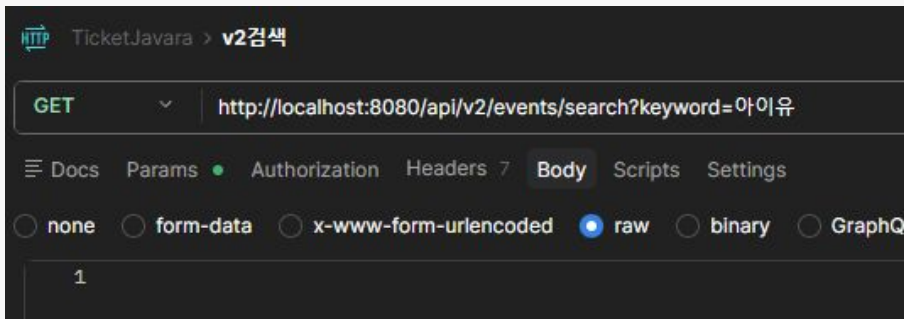
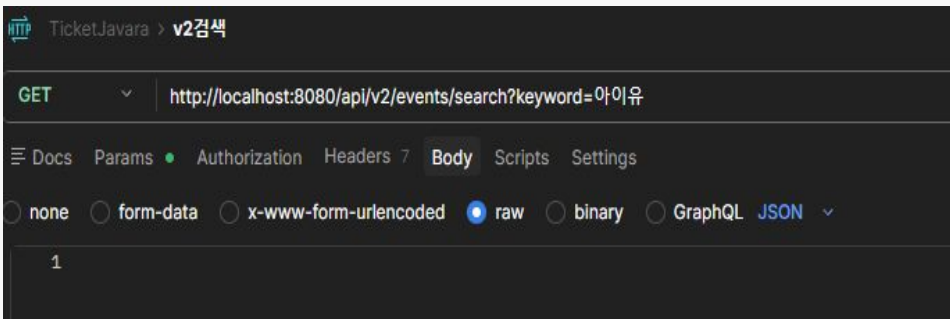
검색 - 캐싱 3단계 진화 이유 - 2(v2 Redis)



v2 Redis 캐시 Miss



v2 : Redis 캐시 히트



예매 - ACTIVE_BOOKING 테이블을 따로 둔 이유 - 1



문제: MySQL 조건부 유니크 인덱스(Partial Index) 미지원

- WHERE status = 'CONFIRMED' 조건부 UNIQUE INDEX → MySQL 미지원
- SEAT.status 컬럼 방식 → 상태 UPDATE 경쟁 + DB 레벨 중복 차단 불가



선택: ACTIVE_BOOKING 분리 + seat_id PK

- 두 번째 INSERT 자체가 DB 레벨에서 실패 → 중복 확정 물리적 차단
- seat_id를 PK로 사용 → 별도 UNIQUE INDEX 없이 보장
- ACTIVE_BOOKING이 없는 이상 CONFIRMED 좌석 중복 불가

예매 - ACTIVE_BOOKING 테이블을 따로 둔 이유 - 1



ORDER_ITEM 제거

BOOKING이 order_id +
original_price 직접 보유
→ JOIN Depth 감소,
MVP 개발 속도 유리



BOOKING = 티켓 검임

별도 TICKET 테이블 대신
ticket_code 컬럼 추가
→ ticketId =
booking_id로 단순화



이중 방어선 구조

Redis 분산락 = 1차 방어선
ACTIVE_BOOKING
PK = 2차 방어선
→ Redis 장애에도 DB가 막음

예매 - 왜 Lettuce SETNX를 골랐고 어떻게 구조화 했는지

Lua Script 원자화

- UUID 검증 + DEL을 한 번의 스크립트로 처리
- GET -> DEL 사이에 다른 스레드가 끼어드는 것을 원천 차단
- > “내 락만 해제된다”는 것을 보장

HoldLockFacade 분리

- @Transactional 메서드 안에서 락을 해제하면 COMMIT 전에 락이 풀리는 버그 발생
- Facade를 별도 Bean으로 분리해서 트랜잭션과 락의 생명주기를 독립적으로 관리
- > Self-invocation 문제도 해결

락 획득 순서 고정

- 1) 좌석 Hold 락 -> 2) 쿠폰 사용 락 -> 3) DB 트랜잭션
- 순서가 다르면 스레드 A와 B가 서로의 락을 기다리는 데드락 발생
- 모든 스레드가 동일한 순서로 락을 획득하면 데드락은 차단됨

검증: 스레드 동시 Hold 테스트

- 락 없음 -> 다수 성공 X
- 락 적용 -> 정확히 1건만 성공

흐름

SETNX 락 획득 -> Facade 실행 -> HoldService
비즈니스 로직 -> Lua Script 락 해제

인프라 - 백업

인프라 - 백업

mysqldump + cron + S3로 매일 새벽 2시 자동 백업



스케줄	매일 새벽 2시 → 0 2 * * *
실행 명령	docker exec ticketjavara-mysql mysqldump ...
저장 위치	s3://ticket-javara-sparta/backups/

현재 EC2 인스턴스에
DOCKER(mysql, redis)를 설치해
사용중인데 EC2에 문제가 발생
때를 대비한 백업 생성입니다.

백업 파일을 EC2 로컬에만 두지
S3에 올리기 때문에, EC2가 완전
날아가도 데이터는 보존됩니다

왜 Lettuce SETNX를 골랐는가

구분	낙관적 락	비관적 락	Lettuce SETNX ✓ 선택
<u>장점</u>	DB 부하 낮음 구현 단순	정확한 순서 보장 강한 일관성	Stateless + Fail Fast 비즈니스 로직 전체 보호
<u>단점</u>	Scale-out 불가 충돌 시 재시도 폭발	단일 DB 의존 연결 점유로 병목 발생	Redis 장애 시 백업 필요 PK로 보완 ✓

핵심 판단 기준

티케팅 = 충돌 많음 + Fail Fast 필요 → Lettuce SETNX가 최적

예매 - **ACTIVE_BOOKING** 테이블을 따로 둔 이유



문제: MySQL 조건부 유니크 인덱스(Partial Index) 미지원

- WHERE status = 'CONFIRMED' 조건부 UNIQUE INDEX → MySQL 미지원
- SEAT.status 컬럼 방식 → 상태 UPDATE 경쟁 + DB 레벨 중복 차단 불가



선택: **ACTIVE_BOOKING** 분리 + seat_id PK

- 두 번째 INSERT 자체가 DB 레벨에서 실패 → 중복 확정 물리적 차단
- seat_id를 PK로 사용 → 별도 UNIQUE INDEX 없이 보장
- ACTIVE_BOOKING이 없는 이상 CONFIRMED 좌석 중복 불가



ORDER_ITEM 제거

BOOKING이 order_id +
original_price 직접 보유
→ JOIN Depth 감소,
MVP 개발 속도 유리



BOOKING = 티켓 검임

별도 TICKET 테이블 대신
ticket_code 컬럼 추가
→ ticketId =
booking_id로 단순화



이중 방어선 구조

Redis 분산락 = 1차 방어선
ACTIVE_BOOKING
PK = 2차 방어선
→ Redis 장애에도 DB가 막음

예매 - 왜 Lettuce SETNX를 골랐고 어떻게 구조화 했는지



Lua Script 원자화

- UUID 검증 + DEL을 한 번의 스크립트로 처리
- GET -> DEL 사이에 다른 스레드가 끼어드는 것을 원천 차단
- → “내 락만 해제된다”는 것을 보장



HoldLockFacade 분리

- @Transactional 메서드 안에서 락을 획득하며 COMMIT 전에 락이 풀리는 버그 발생
- Facade를 별도 Bean으로 분리해서 트랜잭션과 락의 생명주기를 독립적으로 관리
- → Self-invocation 문제도 해결



락 획득 순서 고정

- 1) 좌석 Hold 락 -> 2) 쿠폰 사용 락 -> 3) DB 트랜잭션
- 순서가 다르면 스레드 A와 B가 서로의 락을 기다리는 데드락 발생
 - 모든 스레드가 동일한 순서로 락을 획득하면 데드락은 차단됨



검증: 스레드 동시 Hold 테스트

- 락 없음 -> 다수 성공 X
- 락 적용 -> 정확히 1건만 성공



흐름

SETNX 락 획득 -> Facade 실행 -> HoldService
비즈니스 로직 -> Lua Script 락 해제

예매 - ACTIVE_BOOKING 테이블을 따로 둔 이유

문제: MySQL 조건부 유니크 인덱스(Partial Index) 미지원

- WHERE status = 'CONFIRMED' 조건부 UNIQUE INDEX → MySQL 미지원
- SEAT.status 컬럼 방식 → 상태 UPDATE 경쟁 + DB 레벨 중복 차단 불가

선택: ACTIVE_BOOKING 분리 + seat_id PK

- 두 번째 INSERT 자체가 DB 레벨에서 실패 → 중복 확정 물리적 차단
- seat_id를 PK로 사용 → 별도 UNIQUE INDEX 없이 보장
- ACTIVE_BOOKING이 없는 이상 CONFIRMED 좌석 중복 불가

ORDER_ITEM 제거

BOOKING이 order_id
+ original_price 직접
보유

→ JOIN 덤스 감소,
MVP 개발 속도 유리

BOOKING = 티켓 경입

별도 TICKET 테이블
대신 ticket_code 컬럼
추가

→ ticketId =
booking_id로 단순화

이중 방어선 구조

Redis 분산락 = 1차 방어선
ACTIVE_BOOKING
PK = 2차 방어선
→ Redis 장애에도 DB가 막음

Redis 분산락(1차)

사용자가 좌석을
클릭하면
HoldLockFacade가
Redis에 SETNX
hold:{eventId}:{seatId}
를 시도

HoldService 비즈니스 로직

락을 획득한 1명이
HoldService.process
Hold()를 실행

ACTIVE_BOOKING INSERT(2차)

사용자가 결제를 완료하면
토스페이먼트 승인 후
BookingConfirmService
실행
→ 여기서 2차 방어선 동작
이때 seat_id PK 제약이
두 번째 INSERT를
DB 레벨에서
물리적으로 차단

예매 확정 완료

ACTIVE_BOOKING
INSERT 성공 시
모든 처리가
REQUIRES_NEW
트랜잭션 안에서
원자적으로 커밋됨

예매 - Hold TTL 만료 후 웹훅 수신을 어떻게 처리했는지



문제: Race Condition - TTL 만료 중 결제 완료

- 결제 중 Hold TTL(5분) 만료 → 좌석 해제
- PG사는 결제 완료 웹훅 전송 → 이미 해제된 좌석에 예매 확정 시도 → 데이터 정합성 붕괴

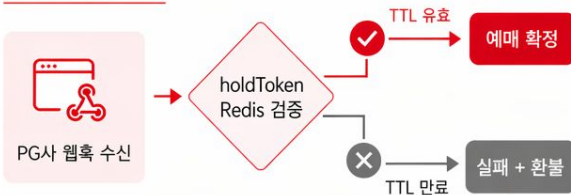


선택: 웹훅 수신 시 holdToken 유효성 재검증

- 웹훅 수신 → Redis holdToken 유효성 재검증
- TTL 유효 → 예매 확정 / TTL 만료 → 예매 실패 처리 + 결제 환불 트리거



흐름



TTL 5분 근거

- 너무 짧음 → 결제 중 만료
- 너무 길면 → 다른 사용자 대기 증가
- 5분 = 결제 완료 여유 + UX 균형

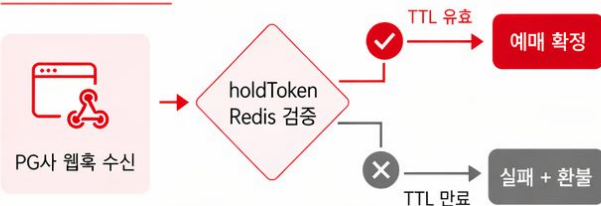


어뷰징 방지 : 1인 최대 4석

- user-hold-count:{userId} 카운터 + TTL 연동
- Set 방식 대신 카운터 방식 → TTL 만료 시 자동 정리



흐름



예매 - 왜 Lettuce SETNX를 골랐고 어떻게 구조화 했는지



검증: 스레드 동시 Hold 테스트

- 락 없음 -> 다수 성공 X
- 락 적용 -> 정확히 1건만 성공



흐름

SETNX 락 획득 -> Facade 실행 -> HoldService
비즈니스 로직 -> Lua Script 락 해제

깔끔하고 현대적인 소프트웨어 아키텍처 전략

3단계 캐싱 진화

1단계 (v1):
캐시 없음: 베이스라인



1단계 (v1):
캐시 없음:
베이스라인

전환 조건



2단계 (v2):
카페인: 로컬 캐시

확장 시 캐시 불일치

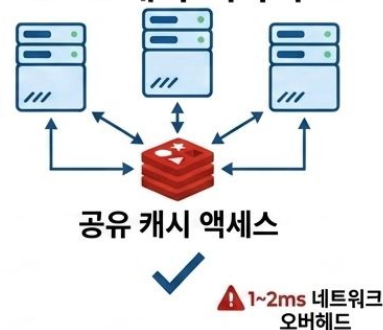


2단계 (v2):
카페인:
로컬 캐시

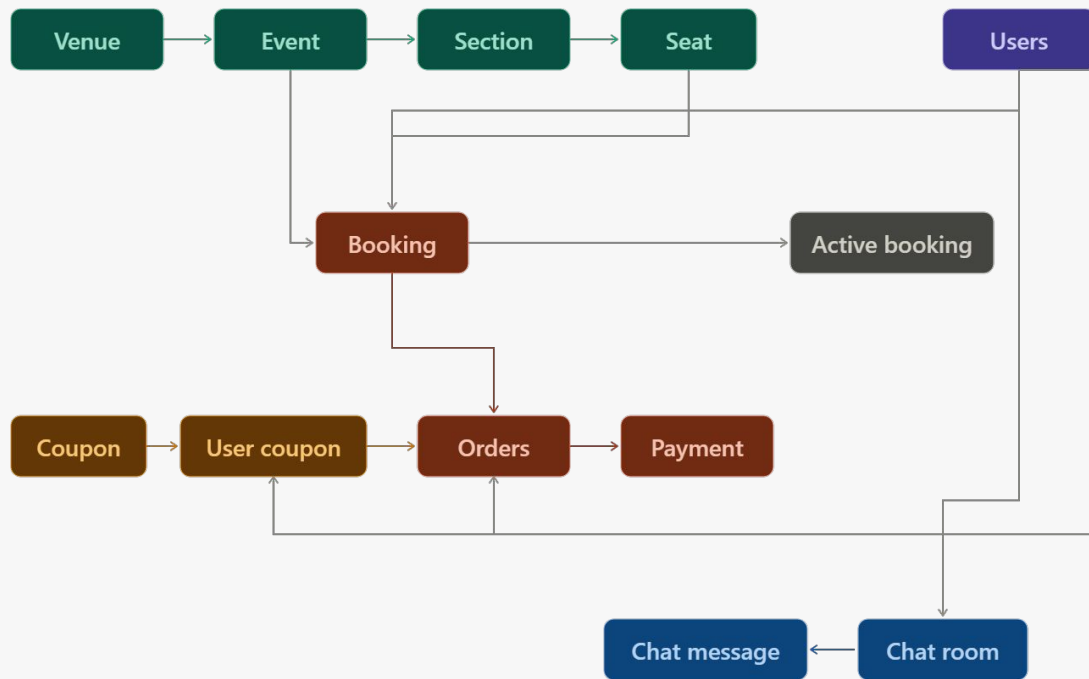
전환 조건



3단계 (v2+):
Redis 캐시-어사이드:



3단계 (v2+):
Redis 캐시-어사이드:
분산 캐시



공연장/구역/좌석

예매/주문/결제

사용자

쿠폰

채팅

활성 예매

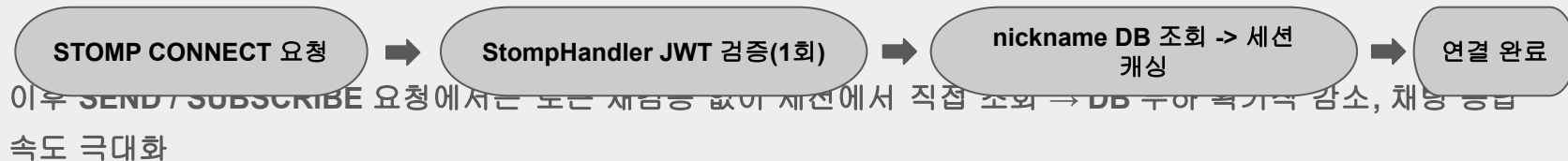
채팅 - WebSocket + STOMP를 선택했는지 <← 제목이 이거 아닌데 바뀌었네요

채팅 - JWT를 STOMP 어느 시점에서 검증했는지

방식	보안	성능	비고
① URL 쿼리 파라미터	✗ 로그·히스토리 토큰 노출	-	보안 위험으로 탈락
② SEND마다 토큰 검증 + DB 조회	✓ 높음	✗ 매 메시지 DB 병목 (N+1 유사)	성능 병목으로 탈락
③ ChannelInterceptor CONNECT 1회 검증	✓ STOMP 헤더로 안전 전달	✓ 세션에 nickname 캐싱 → DB 재조회 0회	보안 + 성능 모두 달성

선택

📌 CONNECT 시점 처리 흐름



왜 Feature Flag로 설계했나?

만약 코드를 직접 교체했다면?

구분	내용
코드 직접 교체	Caffeine → Redis 전환 중 문제 발생 → Git 되돌리기 + 코드 수정 + 재배포 필요 → 운영 중 위험 발생
Feature Flag 	application.yml 값 1개 변경 → 재배포만으로 즉시 전환·롤백. 코드 수정 없음.

application.yml

```
# application.yml
cache:
  provider: caffeine # caffeine | redis (ADR-006)
lock:
```

```
  cache:
    # caffeine | redis (ADR-006: @ConditionalOnProperty로 캐시 구현체
    provider: redis
    lock:
      # lettuce | redisson (ADR-002: @ConditionalOnProperty로 락
      provider: lettuce
```

v1 필수 — 캐시 없음: 기준선 측정

왜 3단계로 나뉘었나? — 각 단계의 한계를 직접 경험해야 다음 단계의 필요성이 생긴다

항목	내용
학습 목표	캐시가 없을 때 DB 부하와 응답 속도를 직접 체감 → 비교 기준선 확보. 이 수치가 없으면 v2 효과를 증명할 수 없다.
한계 →	매 요청마다 MySQL Full Scan 발생. 5,000개 이벤트 기준 응답이 눈에 띄게 느려짐.

▼ v1 대비 응답 50% 이상 단축 확인 후 → 다음 단계 **v2 필수 — Caffeine: 로컬 캐시 적용**

항목	내용
학습 목표	@Cacheable TTL 5분, maximumSize 1000, LRU 적용. JVM 힙 사용 → 응답 50% 이상 단축 체감.
한계 →	Scale-out 시 서버별 캐시가 분리됨 → 서버 A 캐시 히트, 서버 B 캐시 미스 → 불일치 발생.

Caffeine의 Scale-out 한계를 경험한 뒤 → 다음 단계 (도전) **v2+ 도전 — Redis Cache-Aside: 분산 캐시 전환**

항목	내용

캐싱 3단계 진화 & Feature Flag 전환 전략

왜 3단계로
나뉘었나?

v1 : 캐시 없음 기준선 측정

캐시가 없을 때
DB 부하·응답 속도를
직접 체감 — 비교 기준선 확보

한계: 매 요청마다 MySQL Full Scan

V2 : Caffeine 로컬 캐시

@Cacheable TTL 5분
JVM 힙 캐시
→ 응답 50% 이상 단축

한계: Scale-out 시 서버 간
캐시 불일치

V2 : Redis Cache-Aside

전체 서버 캐시 공유
분산 환경 일관성 보장
→ Scale-out 문제 해결

한계: Redis 왕복 1~2ms
추가 비용 발생

왜 Feature Flag로 설계했나?

만약 코드를 직접 교체했다면?

Caffeine → Redis 전환 중 문제 발생
→ Git 되돌리기 + 코드 수정 + 재배포 (운영 위험 ⚠)

Feature Flag 구조

```
# application.yml
cache:
  provider: caffeine # → redis 로 변경만으로 전환
```

코드 변경 Zero

yaml 값 1개 변경으로 전환
재배포만으로 즉시 적용

즉시 롤백 가능

Redis 장애 시 caffeine으로
한 줄 롤백 — 위험 없음

lock과 대칭 구조

lock.provider: lettuce|redisson
동일 패턴 — 팀 학습 일관성

결정: v1(기준선) → v2 Caffeine(필수) → v2 Redis(도전) | 전환 기준: v1 대비 50% 이상 단축 확인 후 | 전환 방식: yaml 한 줄 변경

검색 - 왜 3단계로 나뉘었는지에 대한 고찰

각 단계의 한계를 직접 경험하여 해야 다음 단계의 필요성

v1 필수	캐시 없음 — 기준선 측정
학습 목표	캐시가 없을 때 DB 부하와 응답 속도를 직접 체감 → 비교 기준선 확보. 이 수치가 없으면 v2 효과를 증명할 수 없다.
한계 →	매 요청마다 MySQL Full Scan 발생. 5,000개 이벤트 기준 응답이 눈에 띄게 느려짐.

▼ v1 대비 응답 50% 이상 단축 확인 후 → 다음 단계

v2 필수	Caffeine — 로컬 캐시 적용
학습 목표	@Cacheable TTL 5분, maximumSize 1000, LRU 적용. JVM 힙 사용 → 응답 50% 이상 단축 체감.
한계 →	Scale-out 시 서버별 캐시가 분리됨 → 서버 A 캐시 히트, 서버 B 캐시 미스 → 불일치 발생.

▼ Caffeine의 Scale-out 한계를 직접 경험한 뒤 → 다음 단계 (도전)

v2+ 도전	Redis Cache-Aside — 분산 캐시 전환
학습 목표	전체 서버 캐시 공유, 분산 환경 일관성 보장 → Scale-out 문제 해결. v1/v2/v2+ 3단계 성능 비교 보고서 작성.
한계 →	Redis 왕복 1~2ms 추가 네트워크 비용. 외부 인프라(ElastiCache) 의존성 생김.

영상 봐주셔서
감사합니다.



이벤트 - 왜 DELETED를 별도 status로 뒀는지

현재 이벤트 status 이넘 상태

상태	전환 주체	핵심 이유
ON_SALE	관리자 등록	판매 중
SOLD_OUT	시스템 자동	잔여석 = 0
CANCELLED	관리자 수동	공연 취소
ENDED	스케줄러 자동	날짜 지남
DELETED	관리자 수동	노출 차단